



CS348 INFORMATION SYSTEMS

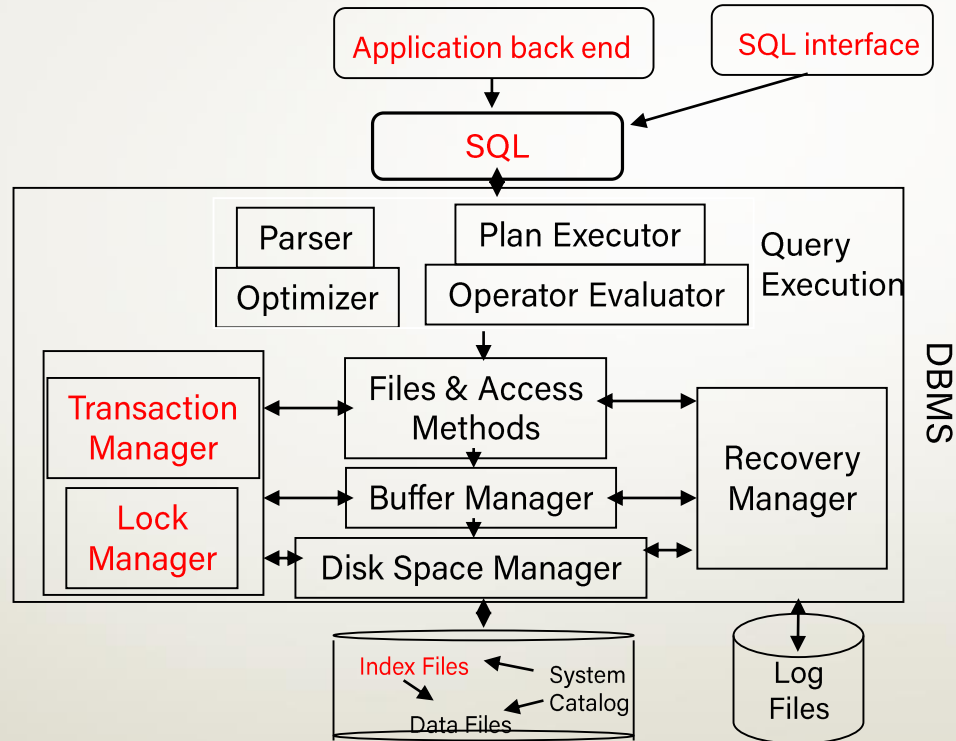
Hisham Benotman

Some slides are adopted with permission from D. Maier and L. Delacambre

Outline

- Using SQL in a Programming Language.

DBMS Architecture



Using SQL in a Programming Language.

- Programmers can:
 - Generate SQL dynamically based on user inputs.
 - Control how data is presented to users.

When are Queries Analyzed?

- Static SQL
- Dynamic SQL

Static SQL

Example in Pro*C

Pro*C code is preprocessed to generate a C/C++ code that is then compiled.

```
{
int a;
/* ... */
EXEC SQL SELECT salary INTO :a
FROM Employee
WHERE SSN=876543210;
/* ... */
printf("The salary is %d\n", a);
/* ... */
}
```

Source: <http://infolab.stanford.edu/~junyang/cs145/or-proc.html>

Dynamic SQL

```
#Python and MySQL Connector
```

```
import mysql.connector
```

```
...
```

```
customPrice = float(input())
```

```
query = """SELECT store, beverage, size, price  
          FROM pricelist  
          WHERE price >=""" + customPrice + ";"
```

```
cursor.execute(query)
```

```
for (store, beverage, size, price) in cursor:  
    print(store, beverage, size, price)
```

When are Queries Analyzed?

- Dynamic SQL
 - Create a string at run time that represents the query
 - Use function calls or methods to pass the string to the DBMS
- Static SQL
 - SQL commands are embedded in program with some kind of special delimiters
 - Use a pre-processor that recognizes the SQL parts of the program
 - Prepare once, execute many times
 - Need a way to pass parameters to query

Dynamic SQL

- Use a collection of functions (Library).
- Examples: JDBC, ODBC, ADO.NET, Psycopg, MySQL Connectors (Python, C, Node.js,...)
- Multiple choices to access a database using Dynamic SQL:
 - a. Text/raw SQL
 - b. Prepared Statement
 - c. Stored Procedure
 - d. Object Relational Mapping (ORM)

Cursors

- Can declare a cursor on a table or query statement (which generates a relation).
- Can *open* a cursor, and repeatedly *fetch* a row (then move the cursor), until all rows have been retrieved.

```
query = "SELECT store, beverage, size, price"\
        " from pricelist;"

cursor.execute(query)

for (store, beverage, size, price) in cursor:
    print(store, beverage, size, price)
```

- May also be possible to modify/delete row pointed to by a cursor.
- Cursor can report conditions (e.g., end of rows)

Examples

Python + MySQL connector

- Text/raw SQL
- Stored Procedure
- Object Relational Mapping (ORM)
- SQL Injection
- Prepared Statement
- Notes:
 - This is a brief introduction and not a complete guide.
 - In the project, check the documentation of your development tools.
 - In the project, use at least two of the following: prepared statements, stored procedures, and ORM.

Raw SQL (1)

Raw SQL (2)

```
16 query = "SELECT store, beverage, size, price from pricelist;"
17
18 cursor.execute(query)
19
20 for (store, beverage, size, price) in cursor:
21     | print(store, beverage, size, price)
22
23 cursor.close()
24 cnx.close()
```

Raw SQL (3) - Output

```
Plaid Pantry Whole Milk 16 1.15
Plaid Pantry 2% Milk 16 1.05
Plaid Pantry Whole Milk 64 3.20
Plaid Pantry Pepsi 12 0.70
Plaid Pantry Diet Pepsi 12 0.70
Plaid Pantry Pepsi 20 0.95
7-11 Whole Milk 8 0.65
7-11 Chocolate Milk 8 0.65
7-11 Whole Milk 16 1.10
7-11 2% Milk 16 1.00
```

HTML Output

```
query = ("SELECT store, beverage, size, price from pricelist;")
cursor.execute(query)
```

```
table = "<html> <table border = '1'>"
```

```
for (store, beverage, size, price) in cursor:
```

```
    table = table + "<tr>\n"
```

```
    table = table + "<td>" + store + "</td>"
```

```
    table = table + "<td>" + beverage + "</td>"
```

```
    table = table + "<td>" + str(size) + "</td>"
```

```
    if(price>=2.0):
```

```
        table = table + "<td style='background-color:#FF0000'>" + str(price)
```

```
    else:
```

```
        table = table + "<td>" + str(price) + "</td>\n"
```

```
    table = table + "</tr>\n"
```

```
table = table + "</table> </html>"
```

Insert

```

11 cursor = cnx.cursor()
12
13 add_beverage = "INSERT INTO pricelist "\
14 |         |         |         |         "(store, beverage, size, price)" \
15 |         |         |         |         "VALUES (%s, %s, %s, %s)"
16
17 beverage_data = ('Walmart', 'Pineapple Juice', 32, 1.5)
18
19 # Insert new beverage
20 cursor.execute(add_beverage, beverage_data)
21
22 # Make sure data is committed to the database
23 cnx.commit()
24 cursor.close()      # close cursor
25 cnx.close()         # close connection

```


Examples

Python + MySQL connector

- ✓ Text/raw SQL
- Stored Procedure
- Object Relational Mapping (ORM)
- SQL Injection
- Prepared Statement

Call a Stored Procedure

```
11 cursor = cnx.cursor()
12 args = ('2% Milk', 16, (0, 'CHAR'), (0, 'CHAR'), (0, 'CHAR'))
13 result_args = cursor.callproc('GetBeverageDetails', args)
14
15 # print results
16 print("Printing procedure output")
17 print(result_args[2])
18 print(result_args[3])
19 print(result_args[4])
```

Examples

Python + MySQL connector

- ✓ Text/raw SQL
- ✓ Stored Procedure
- Object Relational Mapping (ORM)
- SQL Injection
- Prepared Statement

ORM (Object Relational Mapping)

- ORM maps classes in your code to tables in the relational database
 - Your objects will be synced with the table
 - E.g., adding an object in your program -> ORM adds rows in the table

Example Object Relational Mapping (ORM) libraries/frameworks

Django ORM, SQL Alchemy, Hibernate, Sequelize, TypeORM, ...

ORM (Object Relational Mapping)

Example in sqlalchemy ORM library

```
class Store ....
```

```
    __tablename__ = 'stores'
```

```
    name = sqlalchemy.Column(sqlalchemy.String(length=50), primary_key=True)
```

```
    headquarters = sqlalchemy.Column(sqlalchemy.String(length=50))
```

```
    storeType = sqlalchemy.Column(sqlalchemy.String(length=50))
```

ORM (Object Relational Mapping)

Example in sqlalchemy ORM library

```
newStore = Store(name='Meijer', headquarters='Michigan', storeType='Grocery')  
session.add(newStore)
```



New Object



Add my Object to
the Database.
A New Row is
Added

ORM

```
class Store(Base):  
    __tablename__ = 'stores'  
  
    name = sqlalchemy.Column(sqlalchemy.String(length=50), primary_  
headquarters = sqlalchemy.Column(sqlalchemy.String(length=50))  
storeType = sqlalchemy.Column(sqlalchemy.String(length=50))
```

ORM (Insert)

- Insert objects -> ORM Insert rows in DB

```
newStore = Store(name='Meijer', headquarters='Michigan', storeType='Grocery')  
session.add(newStore)
```

```
newStore = Store(name='Walmart', headquarters='Arkansas', storeType='Grocery')  
session.add(newStore)
```

```
newStore = Store(name='ALDI', headquarters='Germany', storeType='Grocery')  
session.add(newStore)
```

```
newStore = Store(name='Kroger', headquarters='Ohio', storeType='Grocery')  
session.add(newStore)
```


ORM (retrieve data)

```
our_store = session.query(Store).filter_by(name='Meijer').first()  
print('\nOur Store:')  
print(our_store)
```

SELECT *

FROM store

WHERE name = 'Meijer'

Limit 1;

ORM (retrieve data)

```
our_stores = session.query(Store).filter_by(storeType='Grocery').all()
print('\nOur Stores:')
print(our_stores)
```

SELECT *

FROM store

WHERE storeType = 'Grocery';

ORM (retrieve data)

```
our_store = session.query(Store.storeType, func.count(Store.name))\
                    .group_by(Store.storeType).all()
print('\nOur Store:')
print(our_store)
```

```
SELECT store.storeType, COUNT(store.name)
FROM store
GROUP BY store.storeType;
```

Examples

Python + MySQL connector

- ✓ Text/raw SQL
- ✓ Stored Procedure
- ✓ Object Relational Mapping (ORM)
- SQL Injection
- Prepared Statement

SQL Injection

- Suppose query is constructed using
 - "select * from instructor where name = " + name + ""
- Suppose the user, instead of entering a name, enters:
 - X' or 'Y' = 'Y
- then the resulting statement becomes:
 - select * from instructor where name = 'X' or 'Y' = 'Y'
 - User could have even used
 - X'; update instructor set salary = salary + 10000; --
- **Always use prepared statements, with user inputs as parameters**
- **Sanitize user input**

Examples

Python + MySQL connector

- ✓ Text/raw SQL
- ✓ Stored Procedure
- ✓ Object Relational Mapping (ORM)
- ✓ SQL Injection
- ✓ Prepared Statement

- Notes:

- This is a brief introduction and not a complete guide.
 - In the project, check the documentation of your development tools.
- In the project, use at least two of the following: prepared statements, stored procedures, and ORM.